Final Paper Stage 2

DATASCI 271

Kathryn Camp Killian Carter Ariel Gholar David Griffin

Imports & Data

The following packages are used in our work:

```
library(dplyr)
library(tidyr)
library(readr)
library(lubridate)
library(ggplot2)
library(urca)
library(changepoint)
library(knitr)
library(tibble)
```

The following code uses data generated in the first project submission:

```
read_quarterly <- function(series, name) {
  read_csv(paste0("", series, ".csv"), show_col_types = FALSE) |>
    mutate(observation_date = tsibble::yearmonth(observation_date)) |>
    tsibble::as_tsibble(index = observation_date) |>
    filter(month(observation_date) %in% c(3, 6, 9, 12)) |>
    mutate(Quarter = tsibble::yearquarter(observation_date)) |>
    tsibble::as_tsibble(index = Quarter) |>
    transmute(date = as.Date(observation_date), !!name := .data[[series]])
}

# All data 1960 Q1 onward
quarterly_data <- read_quarterly("CPIAUCSL", "CPI") |>
  inner_join(read_quarterly("UNRATE", "Unemployment"), by = c("Quarter", "date")) |>
  inner_join(read_quarterly("TB3MS",  "Treasury"),     by = c("Quarter", "date")) |>
  mutate(Inflation = 400 * log(CPI / lag(CPI))) |>
  tsibble::filter_index("1960 Q1" ~ .) |>
  select(Quarter, date, CPI, Inflation, Unemployment, Treasury)
```

```r
# 1960 Q1 through 1999 Q4
initial_data <- quarterly_data |> tsibble::filter_index(~ "1999 Q4")
```

4.2 Assess stationarity and structural change

We use the initial period, 1960Q1–1999Q4 (160 quarters), for every test in this section. Figure 1 shows the three series, with the segment means from a change-point analysis of inflation described below, and Figure 2 shows their autocorrelation functions.

```r
# data definitions
ar_quarterly <- quarterly_data
ar_initial <- initial_data
ar_series_labels <- c(Inflation    = "CPI inflation (%, annualized)",
                      Unemployment = "Unemployment rate (%)",
                      Treasury     = "3-month Treasury bill rate (%)")
ar_f    <- function(x, d = 2) formatC(x, format = "f", digits = d)
# cpt.mean() assumes unit noise variance, so we divide by a robust estimate
# of the quarter-to-quarter noise scale before running PELT.
ar_noise_scale <- function(x) mad(diff(x)) / sqrt(2)

ar_segments_for <- function(x, dates, scale = 1) {
  cp    <- cpt.mean(x / scale, method = "PELT", penalty = "BIC")
  ends  <- c(cpts(cp), length(x))
  start <- c(1, head(ends, -1) + 1)
  tibble(start = dates[start], end = dates[ends],
         mean = param.est(cp)$mean * scale)
}

ar_pi_scale  <- ar_noise_scale(ar_initial$Inflation)
ar_segs      <- ar_segments_for(ar_initial$Inflation, ar_initial$date, ar_pi_scale)
ar_n_raw     <- nrow(ar_segments_for(ar_initial$Inflation, ar_initial$date)) - 1  # Stage 1
ar_n_scaled  <- nrow(ar_segs) - 1
ar_n_other   <- sapply(c("Unemployment", "Treasury"), function(v)
  nrow(ar_segments_for(ar_initial[[v]], ar_initial$date,
                       ar_noise_scale(ar_initial[[v]]))) - 1)

ar_seg_mean <- function(from, to) ar_segs$mean[ar_segs$start >= as.Date(from) &
                                               ar_segs$end <= as.Date(to)]
ar_high <- ar_seg_mean("1973-01-01", "1981-09-30")
ar_low  <- ar_seg_mean("1981-10-01", "1999-12-31")
# The prose names the first two segments and then these two groups, so every
# segment must fall in exactly one of them.
stopifnot(length(ar_high) + length(ar_low) + 2 == nrow(ar_segs))
ar_join_and <- function(x) if (length(x) < 2) x else
  paste0(paste(head(x, -1), collapse = ", "), " and ", tail(x, 1))
```

```r
ar_initial |>
  as_tibble() |>
  pivot_longer(c(Inflation, Unemployment, Treasury), names_to = "series") |>
  mutate(series = factor(series, names(ar_series_labels), ar_series_labels)) |>
  ggplot(aes(date, value)) +
  geom_line(linewidth = 0.4) +
  geom_segment(data = ar_segs |>
                 mutate(series = factor("Inflation", names(ar_series_labels),
                                        ar_series_labels)),
               aes(x = start, xend = end, y = mean, yend = mean),
               colour = "firebrick", linetype = "dashed", linewidth = 0.6) +
  facet_wrap(~ series, ncol = 1, scales = "free_y") +
  labs(x = NULL, y = NULL,
       title = "Inflation, unemployment and the Treasury bill rate",
       subtitle = "Quarterly, 1960Q1 to 1999Q4, with PELT segment means for inflation")
```

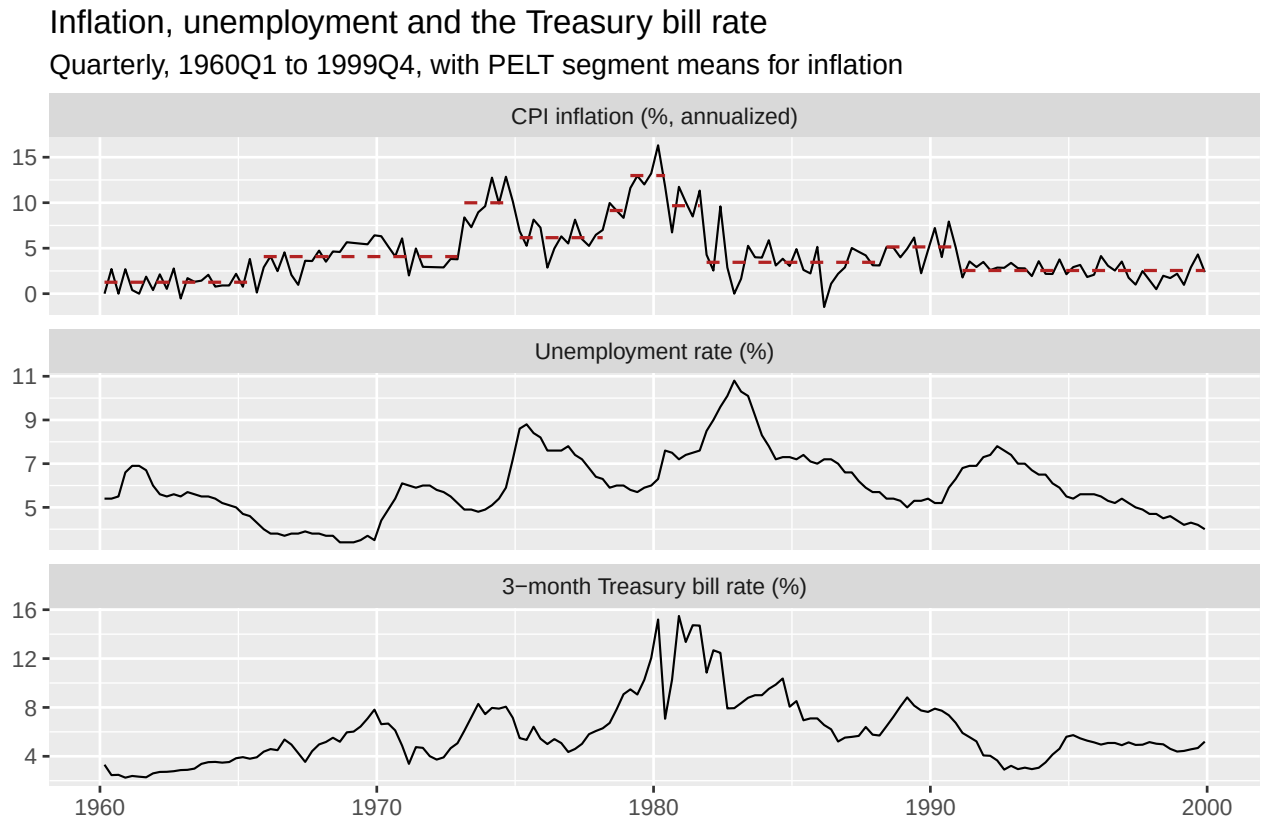


Figure 1: The three series, 1960Q1–1999Q4. Dashed red lines in the inflation panel are PELT segment means after rescaling inflation by its noise scale.

```r
ar_acf_df <- bind_rows(lapply(names(ar_series_labels), function(v) {
  a <- acf(ar_initial[[v]], lag.max = 20, plot = FALSE)
  tibble(series = v, lag = a$lag[-1], acf = a$acf[-1])
})) |> mutate(series = factor(series, names(ar_series_labels), ar_series_labels))
```

```r
ggplot(ar_acf_df, aes(lag, acf)) +
  geom_hline(yintercept = 0, linewidth = 0.3) +
  geom_hline(yintercept = c(-1, 1) * 1.96 / sqrt(nrow(ar_initial)),
             linetype = "dashed", colour = "steelblue") +
  geom_segment(aes(xend = lag, yend = 0)) +
  facet_wrap(~ series, nrow = 1) +
  labs(x = "Lag (quarters)", y = "ACF",
       title = "Autocorrelation of the series in levels, 1960Q1 to 1999Q4")

ar_acf_at <- function(v, k) acf(ar_initial[[v]], lag.max = k, plot = FALSE)$acf[k + 1]
```

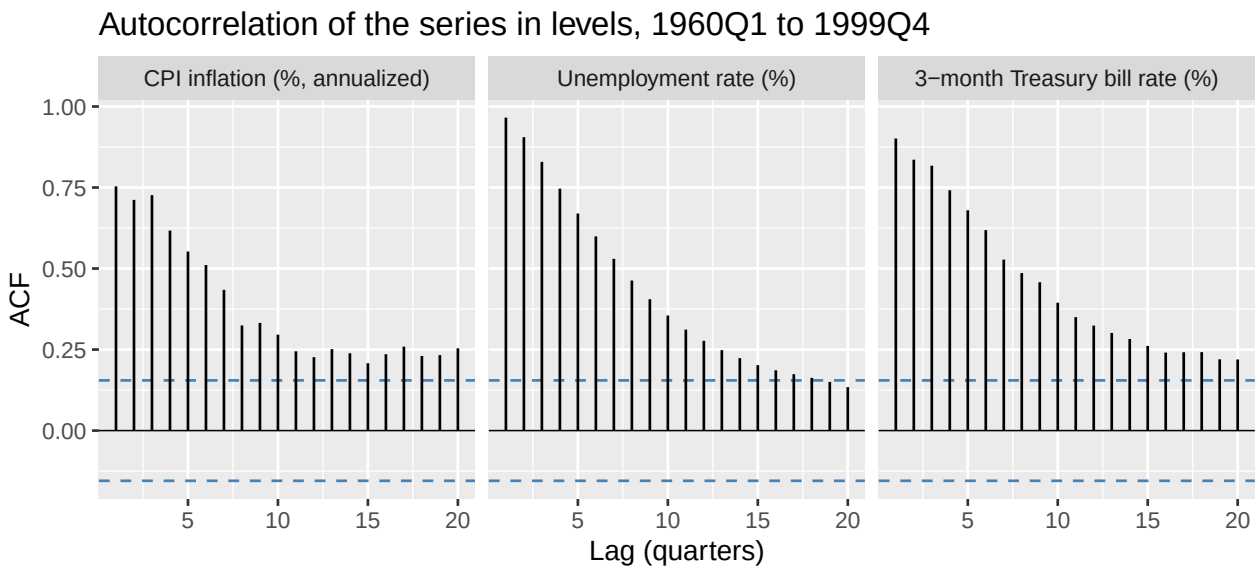


Figure 2: Sample ACFs of the three series in levels, lags 1–20. Dashed lines are approximate 95% bounds.

What the plots show. Inflation rises from about 1% in the early 1960s to double digits in 1974 and 1979–80, then falls back after 1981. Its ACF starts at 0.75 and is still 0.32 at lag 8, so dependence is persistent, but it decays steadily rather than staying near one. Unemployment and the Treasury bill rate are more persistent still, with lag-1 autocorrelations of 0.97 and 0.90. Unemployment moves in long cyclical swings, and the bill rate tracks inflation upward before spiking during the 1979–82 Volcker tightening. None of the three series returns quickly to a single fixed level.

Unit-root tests. For each series we run the augmented Dickey–Fuller (ADF) test with a constant and no trend, since none of these rates has an economic reason to trend deterministically. We choose the augmentation lag from 0–4 by BIC, comparing all five regressions on the same estimation sample so the BIC values are comparable, and then run `urca::ur.df()` at the selected lag. We don't use the `selectlags = "BIC"` option in `ur.df()` here, since it only searches lags 1 through the maximum and can't select 0. The KPSS test uses a constant (`type = "mu"`) and the short Newey–West bandwidth of 4 lags, and we repeat it with the long bandwidth of 13 lags as a robustness check. The ADF null hypothesis is a unit root, and the KPSS null hypothesis is stationarity. When

the level results support a unit root, we repeat both tests on the first difference. Table 1 collects the results.

```
ar_adf_lag_bic <- function(x, max_lag = 4) {
  dx  <- diff(x)
  idx <- (max_lag + 1):length(dx)          # common sample for every lag
  bic <- sapply(0:max_lag, function(k) {
    X <- cbind(level = x[idx],
               if (k > 0) sapply(1:k, function(j) dx[idx - j]))
    BIC(lm(dx[idx] ~ X))
  })
  which.min(bic) - 1
}

ar_run_tests <- function(x) {
  k        <- ar_adf_lag_bic(x)
  adf_fit  <- ur.df(x, type = "drift", lags = k)
  kpss_fit <- ur.kpss(x, type = "mu", lags = "short")
  kpss_lng <- ur.kpss(x, type = "mu", lags = "long")
  tibble(lag = k,
         adf = adf_fit@teststat[1, "tau2"],
         adf_cv = adf_fit@cval["tau2", "5pct"],
         kpss = kpss_fit@teststat, kpss_cv = kpss_fit@cval[1, "5pct"],
         kpss_cv1 = kpss_fit@cval[1, "1pct"],
         kpss_long = kpss_lng@teststat)
}

ar_ur_tbl <- bind_rows(lapply(names(ar_series_labels), function(v) {
  lev <- ar_run_tests(ar_initial[[v]]) |> mutate(series = v, form = "Level")
  if (lev$adf < lev$adf_cv) return(lev)  # unit root rejected: no differencing
  bind_rows(lev, ar_run_tests(diff(ar_initial[[v]])) |>
              mutate(series = v, form = "First difference"))
})) |>
  mutate(adf_reject = adf < adf_cv, kpss_reject = kpss > kpss_cv,
         kpss_long_reject = kpss_long > kpss_cv)

ar_g    <- function(v, form, col) ar_ur_tbl[[col]][ar_ur_tbl$series == v & ar_ur_tbl$form == fo
ar_lv   <- ar_ur_tbl |> filter(form == "Level")
ar_dfs  <- ar_ur_tbl |> filter(form != "Level")
```

In levels, no ADF statistic reaches the 5% critical value of -2.88, and every short-bandwidth KPSS statistic exceeds its 5% critical value of 0.463, though none reaches the 1% value of 0.739. In first differences, both tests indicate stationarity for all three series. The KPSS rejections are also sensitive to the bandwidth. With the long bandwidth, the level statistics fall to 0.235, 0.250 and 0.289, and none reject. The long bandwidth trades power for size, so this doesn't show the series are stationary, but it does mean the KPSS evidence against stationarity is weak for every series.

Structural change. Applied to raw inflation, PELT finds the 17 breaks reported in Stage 1, about

one every 9 quarters, which looks more like PELT tracking a wandering series than like a few regimes. The reason is that `cpt.mean()` assumes unit noise variance, while inflation's quarter-to-quarter noise has a standard deviation of about 1.43 points, so the BIC penalty is too small for the scale of the data. We therefore divide inflation by a robust estimate of its noise scale, $\mathrm{MAD}(\Delta\pi_t)/\sqrt{2}$, before running PELT, which changes the Stage 1 settings. The rescaled analysis finds 9 breaks (Figure 1). They describe inflation near 1.3% through 1965, about 4.1% through 1972, a high and volatile 1973–81 with segment means between 6.2% and 13.0%, and a lower level after 1981, with segment means of 3.4%, 5.1% and 2.5%. These shifts matter for the tests, since a series that is stationary around a mean that shifts a few times can make the ADF test fail to reject and the KPSS test reject (Perron 1989), which is exactly the pattern in levels above. Ignoring the shifts could lead us to treat inflation as having a unit root when it may instead be reverting to a slowly changing level. We don't use PELT for unemployment or the bill rate. Their quarter-to-quarter changes are small relative to their slow swings, so even after the same rescaling PELT splits them into 28 to 31 segments, which describes persistence rather than separate regimes.

```
ar_dec <- function(x) ifelse(x, "reject", "do not reject")
ar_ev  <- function(v, form) paste0(ar_f(ar_g(v, form, "adf")), ", lag ", ar_g(v, form, "lag"),
                                   " (", ar_dec(ar_g(v, form, "adf_reject")), ")")

tibble(
  Series = c("CPI inflation", "Unemployment", "Treasury bill rate"),
  `ADF evidence` = sapply(names(ar_series_labels), function(v)
    paste0("Level ", ar_ev(v, "Level"), ". Difference ",
           ar_ev(v, "First difference"), ".")),
  `KPSS evidence` = sapply(names(ar_series_labels), function(v)
    paste0("Level ", ar_f(ar_g(v, "Level", "kpss"), 3), " (",
           ar_dec(ar_g(v, "Level", "kpss_reject")), "), long ",
           ar_f(ar_g(v, "Level", "kpss_long"), 3), " (",
           ar_dec(ar_g(v, "Level", "kpss_long_reject")), "). Difference ",
           ar_f(ar_g(v, "First difference", "kpss"), 3), " (",
           ar_dec(ar_g(v, "First difference", "kpss_reject")), ").")),
  `Structural-change evidence` = c(
    paste(ar_n_scaled, "mean breaks after rescaling. Low early 1960s, high 1973-81, lower afte
    "No discrete regimes. Long cyclical swings.",
    "No discrete regimes. Rises with inflation, 1979-82 spike."),
  Classification = c("Unresolved", "$I(1)$", "$I(1)$")
) |>
  kable()
```

Table 1: Stationarity summary, 1960Q1–1999Q4. ADF entries are τ statistics with the BIC-selected lag, and KPSS entries are test statistics with the short bandwidth unless marked long. 5% critical values are -2.88 (ADF) and 0.463 (KPSS).

Series	ADF evidence	KPSS evidence	Structural-change evidence	Classification
CPI inflation	Level -2.27, lag 2 (do not reject). Difference -15.24, lag 1 (reject).	Level 0.500 (reject), long 0.235 (do not reject). Difference 0.075 (do not reject).	9 mean breaks after rescaling. Low early 1960s, high 1973–81, lower after 1981.	Unresolved
Unemployment	Level -2.38, lag 1 (do not reject). Difference -7.41, lag 0 (reject).	Level 0.541 (reject), long 0.250 (do not reject). Difference 0.098 (do not reject).	No discrete regimes. Long cyclical swings.	$I(1)$
Treasury bill rate	Level -2.44, lag 3 (do not reject). Difference -7.23, lag 2 (reject).	Level 0.646 (reject), long 0.289 (do not reject). Difference 0.075 (do not reject).	No discrete regimes. Rises with inflation, 1979–82 spike.	$I(1)$

Classification. We classify unemployment and the Treasury bill rate as $I(1)$. The ADF test never rejects, differencing removes the persistence, and their lag-1 autocorrelations of 0.97 and 0.90 decay only slowly, so a unit root is the simpler description even though the KPSS evidence is weak. Both rates are bounded in the long run, so we read $I(1)$ as a description of their behavior over this sample rather than a claim about their true process. We classify inflation as unresolved. Its unit-root tests look like those of the two rates, but its persistence is much lower, with a lag-1 autocorrelation of 0.75 that falls to 0.32 by lag 8, and its mean shifts at identifiable dates that group into three broad regimes, a low early 1960s, a high 1973–81 and a lower level after 1981. A series that is stationary around a shifting mean produces the same test results, while the two rates move too persistently for a few mean shifts to explain them. That is why the AR models in 4.4.3 and 4.4.4 are worth comparing against the gap models in 4.4.5, since the AR models impose one fixed long-run mean and the gap models track a local one. The forecasts will show which description of inflation works better out of sample.

4.3. Step 2: Set up the expanding-window forecast experiment

We implement the expanding-window experiment with two functions that every model section shares, so all models are evaluated on identical origins, samples and targets. The function `run_expanding_window` steps through the forecast origins from 1999Q4 to 2019Q3. At each origin T, it cuts the quarterly data to 1960Q1–T and passes only that sample to each model. A model is written as a function `f(data, H)` that returns its forecast of inflation in quarter T + H.

The framework skips any forecast whose target falls after 2019Q4, which gives exactly the 80, 77 and 73 forecasts per model required at each horizon. It stops with an error if a model fails to return a single finite forecast. The output has one row per model, origin and horizon, recording the origin, target quarter, horizon, model and forecast.

The function `attach_actuals` is a separate step, run only after all forecasts are made. It joins realized inflation by target quarter and computes the error $e = \pi_t - \hat{\pi}_{t|t-h}$.

```
# Returns one row per model, origin and horizon (origin, target, h, model,
# forecast) for every target from the first origin through last_target:
# 80, 77 and 73 forecasts per model at h = 1, 4 and 8.
run_expanding_window <- function(data, models,
                                 first_origin = "1999 Q4",
                                 last_target  = "2019 Q4",
                                 horizons     = c(1, 4, 8)) {
  horizons <- as.integer(horizons)
  quarters <- data[[tsibble::index_var(data)]]   # sorted, one row per quarter
  first <- which(quarters == tsibble::yearquarter(first_origin))
  last  <- which(quarters == tsibble::yearquarter(last_target))

  out <- list()
  for (i in first:(last - min(horizons))) {    # i is the row of origin T
    data_T <- data[seq_len(i), ]               # 1960 Q1 through T, nothing later
    for (H in horizons[i + horizons <= last]) {
      for (m in names(models)) {
        fc <- models[[m]](data_T, H)
        if (length(fc) != 1 || !is.finite(fc)) {
          stop(sprintf("'%s' at origin %s, H = %d did not return one finite number.",
                       m, format(quarters[i]), H), call. = FALSE)
        }
        out[[length(out) + 1]] <- tibble(origin   = quarters[i],
                                         target   = quarters[i + H],
                                         h        = H,
                                         model    = m,
                                         forecast = as.numeric(fc))
      }
    }
  }
  bind_rows(out) |> arrange(match(model, names(models)), h, origin)
}
# Attaches realized inflation for each target quarter and the forecast error
# e = actual - forecast (positive = the model underpredicted). Run this only
# after the forecasts are made; actuals never enter their construction.
attach_actuals <- function(forecasts, data) {
  realized <- tibble(target = data[[tsibble::index_var(data)]],
                     actual = data$Inflation)
  forecasts |>
    left_join(realized, by = "target") |>
    mutate(error = actual - forecast)
}
```

4.4. Step 3: Implement the core forecasting models

4.4.1 Random-walk model

The random walk without drift treats each quarter's inflation as last quarter's inflation plus an unpredictable shock:

$$\pi_t = \pi_{t-1} + \varepsilon_t, \qquad E(\varepsilon_t) = 0, \qquad Var(\varepsilon_t) = \sigma_\varepsilon^2.$$

Since future shocks have mean zero, the best forecast at origin T is the most recent observation at every horizon:

$$\hat{\pi}^{RW}_{T+h|T} = \pi_T, \qquad h = 1, \dots, 8.$$

In code, `fc_rw(data, H)` returns `data$Inflation[nrow(data)]` for every horizon H, using only the sample through T. The model has no parameters to estimate, so "re-estimating" it at each origin just means reading off the latest inflation rate, which makes it the simplest reference in the comparison. It never reverts toward any mean, which is where it parts company with a stationary AR, and it reacts fully to whatever happened in the last quarter, including noise.

```
fc_rw <- function(data, H) {
  data$Inflation[nrow(data)]
}
```

4.4.2 Four-quarter-average benchmark

The four-quarter-average benchmark is used as a benchmark to be beaten by more complex models. In this naive method, the four most recently observed quarters are averaged to remove seasonal variation, and this average is used as the estimate for all future forecasts. This estimate can be mathematically represented as

$$\hat{\pi}^{4Q}_{T+h|T} = \frac{1}{4}\sum_{j=0}^{3} \pi_{T-j},$$

where T is the most recently observed quarter and h is the forecast horizon. We implement the four-quarter-average benchmark with the following R function:

```
fc_4q <- function(data, H) {
  mean(tail(data$Inflation, 4))
}
```

4.4.3 AR(p) model

The AR(p) model uses past observations of inflation to forecast future values. In an AR(p) process, rather than assigning equal weight to the previous four observations (as the four-quarter-average), the AR(p) model allows inflation to depend on its own lagged values, with the contribution of each lag estimated from the data. For each forecast origin, we estimate AR(1) thorugh AR(4) using maximum likelihood using the available expanding-window sample. We then select the lag order with the lowest Bayesian Information Criterion (BIC), which penalizes additional parameters while accounting for model fit. The AR(p) model is given by:

$$\pi_t = c + \sum_{j=1}^{p} \phi_j \pi_{t-j} + \varepsilon_t$$

where c is the intercept, ϕ_j are the AR coefficients, and ε_t is the error. To enable BIC values to be comparable across lag orders, we estimate all four candidate models using the same observations within each forecast window. We implement the AR(p) model with the following R function:

```
fc_arp <- function(data, H) {
  x <- data$Inflation
  # estimate AR(1) through AR(4)
  models <- lapply(1:4, function(p) {
    stats::arima(
      x,
      order = c(p, 0, 0),
      include.mean = TRUE,
      method = "ML")
  })
  # select the model with the lowest BIC
  bic_values <- sapply(models, stats::BIC)
  best_model <- models[[which.min(bic_values)]]
  # forecast the next h quarters
  as.numeric(stats::predict(best_model, n.ahead = H)$pred[H]
  )
}
```

4.4.4 Direct AR model

The direct AR model forecasts inflation by estimating a separate regression for each forecast horizon. In the AR(p) model above, forecasts beyond one period are captured recursively using the estimated coefficients. In contrast, the direct AR model estimates a separate model for each forecast horizon that relates future inflation to inflation at the forecast origin. The model is given by:

$$\pi_{t+h} = c_h + \sum_{j=0}^{p_{h-1}} \phi_{h,j} \pi_{t-j} + \varepsilon_{t+h}$$

At each forecast origin T, we estimate a separate regression for each horizon h by OLS using only observations for which both the predictors and the horizon-specific response is already known. For

each horizon, we consider the lag orders $p_h \in 1, 2, 3, 4$ and select the model with the lowest BIC. The selected model is then used to produce a h-step ahead forecast directly from teh observations available at forecast origin (T), without using forecasts from earlier horizons as input. We implement the direct AR model with the following R function:

```r
fc_direct_ar <- function(data, H) {
  x <- data$Inflation
  n <- length(x)
  # only responses observed through origin T
  t <- 4:(n-H)
  # construct horizon-specific regression data
  reg_data <- tibble::tibble(
    y = x[t+H],
    lag0 = x[t],
    lag1 = x[t-1],
    lag2 = x[t-2],
    lag3 = x[t-3]
  )
  # estimate candidate orders p= 1, 2, 3, 4
  fits <- lapply(1:4, function(p) {
    predictors <- paste0("lag", 0:(p-1))
    formula <- stats::reformulate(
      predictors, response = "y"
    )
    stats::lm(formula, data = reg_data)
  })
  # select order with lowest BIC
  bic_values <- sapply(fits, stats::BIC)
  best_p <- which.min(bic_values)
  # latest observations available at origin T
  new_data <- tibble::tibble(
    lag0 = x[n],
    lag1 = x[n-1],
    lag2 = x[n-2],
    lag3 = x[n-3]
  )
  # forecast inflation directly at T + h
  as.numeric(
    stats::predict(fits[[best_p]], newdata = new_data)
  )
}
```

4.4.5 Local mean by exponential smoothing

Following Faust and Wright (2013), we estimate a slowly changing local mean τ_t by simple exponential smoothing with $\alpha = 0.05$, a weight the project fixes rather than estimates. We start the recursion at the first observation and update one quarter at a time:

$$\tau_{1960Q1} = \pi_{1960Q1}, \qquad \tau_t = 0.95\,\tau_{t-1} + 0.05\,\pi_t, \qquad g_t = \pi_t - \tau_t.$$

Each new quarter moves the local mean only 5% of the way toward current inflation, so τ_t follows sustained changes in the level but largely ignores one-quarter spikes. Unrolling the recursion shows that τ_t is a weighted average of current and past inflation with weights $0.05(0.95)^j$, which halve roughly every 13.5 quarters. The starting value $\tau_{1960Q1} = \pi_{1960Q1}$ is a single noisy observation, but its weight in τ_t is 0.95^{t-1}, which is 0.13 by 1970Q1 and 0.0003 by the first forecast origin, so the initialization has no practical effect on the local mean during the forecast period. The gap g_t measures how far inflation sits above or below that local level.

```
# 4.4.5 Local mean by simple exponential smoothing --------------------------
# tau_1960Q1 = pi_1960Q1, tau_t = (1 - alpha) * tau_{t-1} + alpha * pi_t,
# with alpha = 0.05 fixed by the project (not estimated).
local_mean <- function(pi, alpha = 0.05) {
  stopifnot(length(pi) > 0, !anyNA(pi))
  tau <- numeric(length(pi))
  tau[1] <- pi[1]
  for (t in seq_along(pi)[-1]) {
    tau[t] <- (1 - alpha) * tau[t - 1] + alpha * pi[t]
  }
  tau
}

# Adds `tau` (local mean) and `gap` (pi_t - tau_t) to a sample.
# Call it inside a model's fc_ function on the origin-T sample it receives,
# which starts in 1960Q1 so tau is initialized at pi_1960Q1. The last row then
# holds tau_T and g_T.
#   Gap AR (4.4.6):            fit the AR to data$gap, forecast = tau_T + g-hat_{T+H}
#   Fixed persistence (4.4.7): forecast = tau_T + 0.46^H * g_T
add_local_mean <- function(data, alpha = 0.05) {
  data$tau <- local_mean(data$Inflation, alpha)
  data$gap <- data$Inflation - data$tau
  data
}
```

```
# Guide example: previous tau 3.0, current inflation 5.0 -> tau 3.1, gap 1.9
ar_ex_tau <- 0.95 * 3.0 + 0.05 * 5.0
stopifnot(isTRUE(all.equal(ar_ex_tau, 3.1)), isTRUE(all.equal(5.0 - ar_ex_tau, 1.9)))

# One-sidedness: tau computed on data through 1999Q4 equals the full-sample
# tau over the same quarters, so tau_T never uses information after T.
ar_hist_sample <- ar_quarterly |> filter(date <= as.Date("2019-12-01"))
ar_full_tau    <- local_mean(ar_hist_sample$Inflation)
ar_origin_tau  <- local_mean(ar_initial$Inflation)
ar_one_sided   <- identical(ar_origin_tau, ar_full_tau[seq_along(ar_origin_tau)])
stopifnot(ar_one_sided)
```

```
ar_lm_hist <- add_local_mean(ar_hist_sample)
ar_tau_at  <- function(qtr) ar_lm_hist$tau[ar_lm_hist$Quarter == tsibble::yearquarter(qtr)]
ar_gap_at  <- function(qtr) ar_lm_hist$gap[ar_lm_hist$Quarter == tsibble::yearquarter(qtr)]
```

The recursion is one-sided because τ_t depends only on π_t and earlier values. At each forecast origin T, `add_local_mean()` runs the update from 1960Q1 through T on the origin's own sample, so τ_T and g_T use nothing after T. As a check, the local mean computed from data through 1999Q4 matches the 1960Q1–1999Q4 portion of the local mean computed through 2019Q4 exactly. The function also reproduces the guide's example, where a previous local mean of 3.0 and current inflation of 5.0 give $\tau_t = 3.1$ and $g_t = 1.9$.

```
ar_lm_hist |>
  as_tibble() |>
  transmute(date,
            `Inflation and local mean` = Inflation,
            tau,
            `Inflation gap` = gap) |>
  pivot_longer(c(`Inflation and local mean`, `Inflation gap`),
               names_to = "panel") |>
  mutate(tau = ifelse(panel == "Inflation and local mean", tau, NA),
         panel = factor(panel, c("Inflation and local mean", "Inflation gap"))) |>
  ggplot(aes(date)) +
  geom_hline(data = tibble(panel = factor("Inflation gap",
                                          c("Inflation and local mean", "Inflation gap")),
                           y = 0),
             aes(yintercept = y), linewidth = 0.3) +
  geom_line(aes(y = value), linewidth = 0.35, colour = "grey35") +
  geom_line(aes(y = tau), linewidth = 0.9, colour = "firebrick", na.rm = TRUE) +
  geom_vline(xintercept = as.Date("1999-12-01"), linetype = "dotted") +
  facet_wrap(~ panel, ncol = 1, scales = "free_y") +
  labs(x = NULL, y = "Percent",
       title = "Local mean (alpha = 0.05) and inflation gap",
       subtitle = "Red line: local mean. Dotted line: first forecast origin, 1999Q4")
```

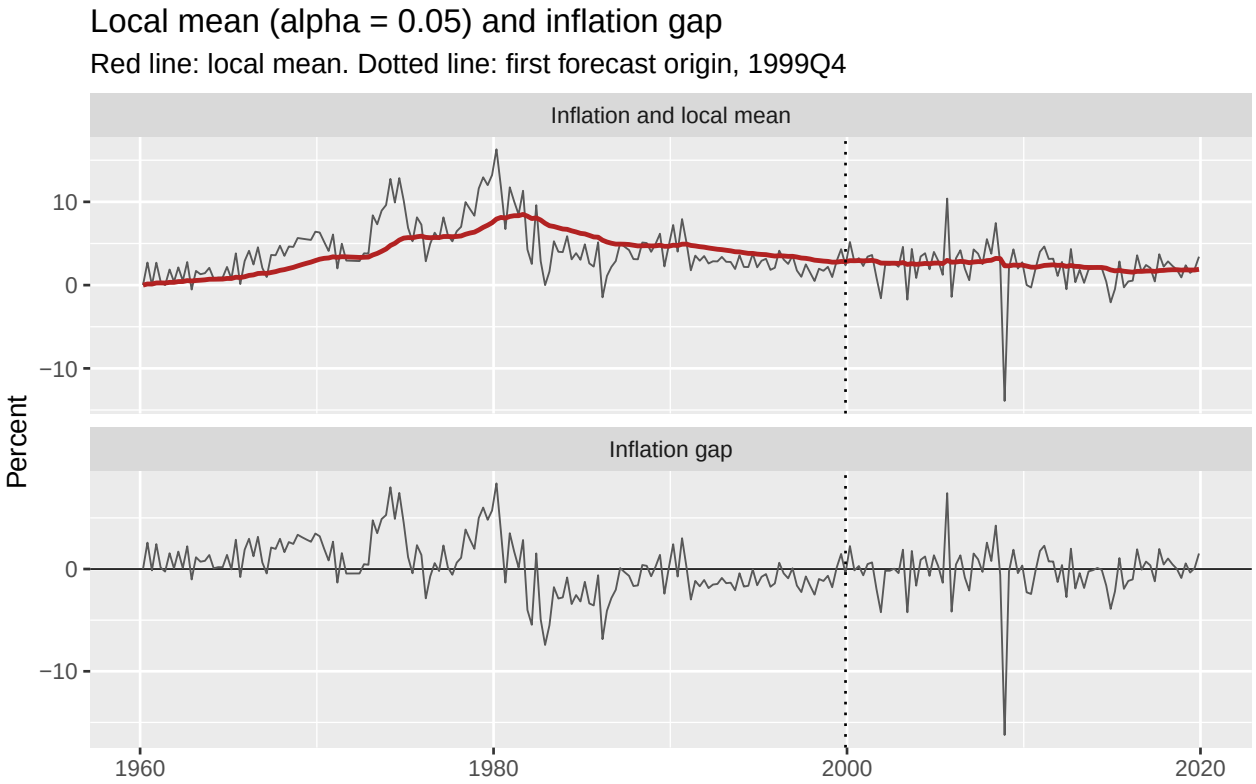


Figure 3: Inflation and its exponentially smoothed local mean (top), and the inflation gap (bottom), 1960Q1–2019Q4. Values through any origin T are the ones available at T.

Figure 3 shows how the local mean behaves. It climbs from 0.8% in 1965Q1 to a peak of 8.5% in 1981 Q3, then declines slowly to 2.8% at the first forecast origin and 1.9% by 2019Q4. It lags turning points by several years, since the weight on new data is small. For example, the gap is 8.4 points in 1980Q1, when inflation surged well above the slowly rising local mean, and the local mean is still above inflation for much of the 1980s. Through the 1990s inflation is lower and steadier than the local mean, so the gap is mostly negative while the local mean drifts down to meet it. By the 2000s the gap is centered close to zero, with an average of -0.22 over 2000Q1–2019Q4. The gap is also less persistent than inflation itself. Its lag-1 autocorrelation over 1960Q1–1999Q4 is 0.66, compared with 0.75 for inflation.

The gap AR (4.4.6) and the fixed-persistence forecast (4.4.7) call `add_local_mean()` on each origin's sample and read τ_T and g_T from the last row. Both forecasts hold the local mean at τ_T, since the future inflation needed to update it isn't observed yet.

4.4.6 Gap AR model

The gap AR model forecasts inflation in two parts: the slowly changing local mean τ_t from 4.4.5, and the inflation gap $g_t = \pi_t - \tau_t$ around it. At each forecast origin, we compute τ_t and g_t from the origin's own sample and fit an autoregression with an intercept to the gap,

$$g_t = a + \sum_{j=1}^{p} \rho_j g_{t-j} + v_t,$$

by OLS for each $p \in \{1, 2, 3, 4\}$, keeping the lag order with the lowest BIC. All four regressions use the same estimation sample, starting in the fifth quarter, so their BIC values are comparable. We then forecast the gap iteratively, feeding each forecast back in as a lag,

$$\hat{g}_{T+k|T} = \hat{a} + \sum_{j=1}^{p} \hat{\rho}_j \, \hat{g}_{T+k-j|T}, \qquad k = 1, \dots, h,$$

where $\hat{g}_{T+k-j|T} = g_{T+k-j}$ whenever $k - j \leq 0$, and add the current local mean back:

$$\hat{\pi}^{GAP}_{T+h|T} = \tau_T + \hat{g}_{T+h|T}.$$

The local mean is held at τ_T over the forecast period, since the future inflation needed to update it isn't observed yet. Comparing this model with the AR(p) model shows whether recentering inflation around a slowly changing level improves forecasts. We implement the gap AR model with the following R function, which uses `add_local_mean()` from 4.4.5:

```r
fc_gap_ar <- function(data, H, max_p = 4) {
  d <- add_local_mean(data)
  g <- d$gap
  n <- length(g)

  idx  <- (max_p + 1):n                                  # common sample for every p
  lags <- sapply(1:max_p, function(j) g[idx - j])        # column j is g_{t-j}
  fits <- lapply(1:max_p, function(p) lm(g[idx] ~ lags[, 1:p, drop = FALSE]))
  p    <- which.min(sapply(fits, BIC))
  b    <- coef(fits[[p]])                                # a, rho_1, ..., rho_p

  path <- tail(g, p)                                     # g_{T-p+1}, ..., g_T
  for (k in seq_len(H)) {
    path <- c(path, b[1] + sum(b[-1] * rev(tail(path, p))))
  }
  d$tau[n] + path[length(path)]
}
```

4.4.7 Fixed-persistence gap forecast

The fixed-persistence gap model uses the local mean (tau_t, see above) together with the inflation gap to forecast inflation. We follow Faust and Wright (2013) and set the persistence coefficient to 0.46 rather than estimating it from the data. The forecast is given by:

$$\hat{\pi}^{FIX}_{T+h|T} = \tau_T + 0.46^h g_T$$

where T is the latest observed quarter and h is the forecast horizon. At each forecast origin, the local mean and inflation gap are calculated from the observations available up to that quarter. The local mean remains constant over the forecast horizon, while the gap gradually approaches zero. Comparing this model to the gap AR model allows us to determine whether estimating the persistence coefficient leads to more accurate forecasts despite the additional estimation uncertainty. We implement the fixed-persistence gap model with the following R function:

```r
fc_fixed_gap <- function(data, H, rho = 0.46) {
  data <- add_local_mean(data)
  tau_T <- tail(data$tau, 1)
  gap_T <- tail(data$gap, 1)

  forecast <- tau_T + rho^H * gap_T

  return(as.numeric(forecast))
}
```

4.4.8 Direct Phillips-curve model

The direct Phillips-curve model builds on the direct AR model by adding the unemployment rate as a predictor of future inflation. A separate regression is fitted for each horizon h, with future inflation regressed on the current unemployment rate and current and lagged inflation. The model is given by:

$$\pi_{t+h} = c_h + \sum_{j=0}^{p_{h-1}} \phi_{h,j}\pi_{t-j} + \lambda_h u_t + \varepsilon_{t+h}$$

At a given forecast origin T, OLS estimation is restricted to observations for which future inflation is already observed. We look to inflation lag orders $p_h \in 1, 2, 3, 4$ and choose from them using BIC. Unemployment is included in all four of these candidate models, and the estimation sample is kept the same across all four. By comparing the forecast errors of the direct Phillips-curve model with those of the direct AR model (which does not include unemployment), we can determine whether unemployment provides useful information about future inflation beyond what is already integrated in current and lagged inflation. We implement the direct Phillips-curve model with the following R function:

```r
fc_direct_phillips <- function(data, H) {
  x <- data$Inflation
  u <- data$Unemployment
  n <- length(x)
  stopifnot(length(u) == n)
  # only responses through origin T
  t <- 4:(n - H)
  # construct horizon-specific regression data
  reg_data <- tibble::tibble(
      y = x[t+H],
      lag0 = x[t],
      lag1 = x[t-1],
      lag2 = x[t-2],
      lag3 = x[t-3],
      UNRATE = u[t]
    )
  # estimate candidate orders p = 1, 2, 3, 4
  fits <- lapply(1:4, function(p) {
      predictors <- c(
        paste0("lag", 0:(p-1)),
        "UNRATE"
      )
      formula <- stats::reformulate(
      predictors, response = "y"
    )
      stats::lm(formula, data = reg_data)
    })
  # select order with lowest BIC
  bic_values <- sapply(fits, BIC)
  best_p <- which.min(bic_values)
  # latest observations available at origin T
  new_data <- tibble::tibble(
    lag0 = x[n],
    lag1 = x[n-1],
    lag2 = x[n-2],
    lag3 = x[n-3],
    UNRATE = u[n]
  )
  # forecast inflation at T + H
  as.numeric(
    stats::predict(fits[[best_p]], newdata = new_data)
    )
  }
```

Expanding window forecast experiment (Step 2)

```
# all models
models <- list(
  "Random walk" = fc_rw,
  "Four-quarter average" = fc_4q,
  "Selected AR(p)" = fc_arp,
  "Direct AR" = fc_direct_ar,
  "Gap AR" = fc_gap_ar,
  "Fixed-persistence gap" = fc_fixed_gap,
  "Direct Phillips-curve" = fc_direct_phillips
)

forecast_results <- run_expanding_window(
  data = quarterly_data,
  models = models,
  first_origin = "1999 Q4",
  last_target = "2019 Q4",
  horizons = c(1, 4, 8)
)

forecast_results <- attach_actuals(
  forecasts = forecast_results,
  data = quarterly_data
)
```

Step 4: Evaluate the forecasts

```
# calculate forecast accuracy by model and horizon
accuracy_results <- forecast_results |>
  dplyr::group_by(model, h) |>
  dplyr::summarise(
    n_h = dplyr::n(),
    RMSE = sqrt(mean(error^2)),
    MAE = mean(abs(error)),
    Mean_error = mean(error), .groups = "drop"
  ) |>
  dplyr::group_by(h) |>
  dplyr::mutate(
    Relative_RMSE = RMSE / RMSE[model == "Four-quarter average"]
    ) |>
      dplyr::ungroup() |>
      dplyr::arrange(h, RMSE)

accuracy_table <- accuracy_results |>
```

```
  dplyr::transmute(
    Model = model,
    Horizon = h,
    n_h,
    RMSE,
    `Relative RMSE` = Relative_RMSE,
    MAE,
    `Mean error` = Mean_error
  )

knitr::kable(
  accuracy_table,
  digits = 3,
  booktabs = TRUE,
  caption = "Expanding-window forecast accuracy"
)
```

Table 2: Expanding-window forecast accuracy

Model	Horizon	n_h	RMSE	Relative RMSE	MAE	Mean error
Fixed-persistence gap	1	80	3.002	0.988	1.873	-0.118
Four-quarter average	1	80	3.039	1.000	1.873	-0.014
Gap AR	1	80	3.058	1.006	1.881	-0.091
Selected AR(p)	1	80	3.079	1.013	1.911	-0.267
Direct AR	1	80	3.088	1.016	1.920	-0.316
Direct Phillips-curve	1	80	3.146	1.035	1.986	-0.339
Random walk	1	80	3.821	1.258	2.442	0.013
Fixed-persistence gap	4	77	2.726	0.933	1.617	-0.302
Gap AR	4	77	2.856	0.978	1.721	-0.265
Four-quarter average	4	77	2.921	1.000	1.751	-0.093
Selected AR(p)	4	77	2.936	1.005	1.813	-0.624
Direct AR	4	77	3.097	1.061	2.001	-0.780
Direct Phillips-curve	4	77	3.107	1.064	2.009	-0.775
Random walk	4	77	3.940	1.349	2.487	-0.057
Fixed-persistence gap	8	73	2.781	0.912	1.688	-0.370
Gap AR	8	73	2.891	0.948	1.862	-0.400
Four-quarter average	8	73	3.049	1.000	2.043	-0.110
Selected AR(p)	8	73	3.063	1.005	2.052	-0.952
Direct AR	8	73	3.236	1.061	2.184	-1.469
Direct Phillips-curve	8	73	3.286	1.078	2.263	-1.581
Random walk	8	73	3.826	1.255	2.553	-0.092

$h = 1$**.** The fixed-persistence gap model has the lowest RMSE of 3.002, slightly outperforming the four-quarter-average benchmark (3.039). Its relative RMSE of 0.988 is a 1.2% improvement. The two models have identical MAE values of 1.873, indicating similar typical forecast errors. The random walk performs worst, with a RMSE of 3.821.

$h = 4$. The fixed-persistence gap model again has the lowest RMSE (2.726), outperforming the benchmark (2.921) by approximately 6.7%. The gap AR model ranks second with a RMSE of 2.856. The fixed-persistence model also has the lowest MAE (1.617), indicating smaller average absolute errors. Its negative mean error (-0.302) suggests a tendency to overpredict inflation (common among all models).

$h = 8$. The fixed-persistence gap model again has the lowest RMSE (2.781), which is a 8.8% improvement over the benchmark (3.049). The gap AR model ranks second with a RMSE of 2.891. The fixed-persistence model also produces the lowest MAE (1.688). However, its mean error of -0.370 indicates continued overprediction (as above, common among all models).

Across all forecast horizons, the fixed-persistence gap model generates the lowest RMSE among the seven evaluated models in this out-of-sample evaluation. Its relative improvement over the four-quarter-average benchmark increases with the forecast horizon. The results suggest that restricting inflation-gap persistence to a fixed coefficient of 0.46 may reduce forecast errors relative to estimating additional autoregressive parameters.

Step 5: Diagnose the leading one-step model

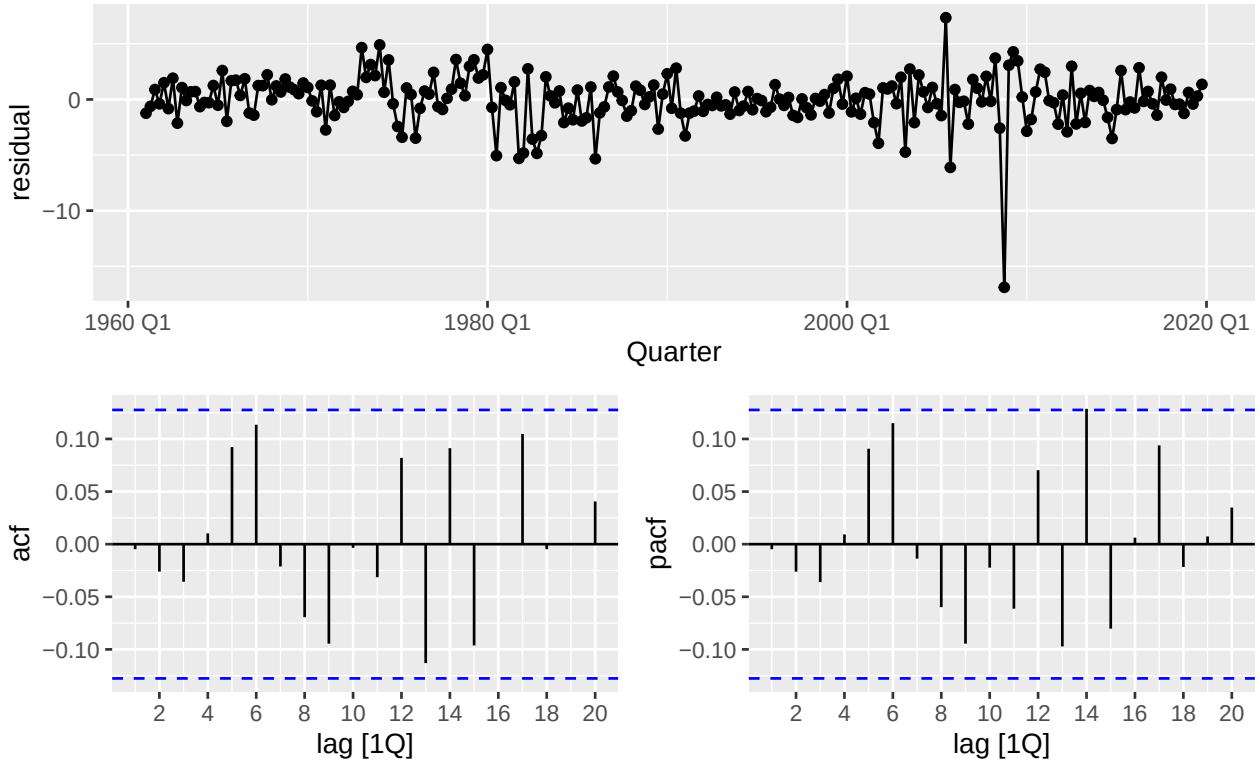
The gap AR model has the lowest $h = 1$ RMSE among the estimated core models, with 3.058. The section below diagnoses the gap AR model.

```
# use data through 2019 Q4
data19 <- quarterly_data |>
  tsibble::filter_index(~ "2019 Q4") |>
  add_local_mean(alpha = 0.05)

# fit Gap AR model using the same sample
g <- data19$gap
n <- length(g)
idx <- 5:n
lags <- sapply(1:4, function(j) g[idx - j])

fits <- lapply(1:4, function(p) {
  lm(g[idx] ~ lags[, 1:p, drop = FALSE])
})
# select AR order using BIC
best_p <- which.min(sapply(fits, BIC))
best_model <- fits[[best_p]]
# extract fitted model residuals
residual_data <- data19[idx, ] |>
  dplyr::mutate(
    residual = as.numeric(residuals(best_model))
  )

# residual time plot plus ACF/PACF
residual_data |>
  feasts::gg_tsdisplay(residual, plot_type="partial", lag_max = 20)
```



Residual Time Plot and ACF/PACF The gap AR model's residuals mostly fluctuate around zero, though several periods show particularly large errors (for example, in the late 2000s). Residual variability also appears to change over time, suggesting possible instability in the error variance. Neither the residual ACF nor PACF show significant spikes at the displayed lags, suggesting that there is no remaining serial dependence (at least based on the time plot and ACF/PACF).

```
lb_result <- stats::Box.test(
  residual_data$residual,
  lag = 12,
  type = "Ljung-Box",
  fitdf = best_p
)
lb_result
```

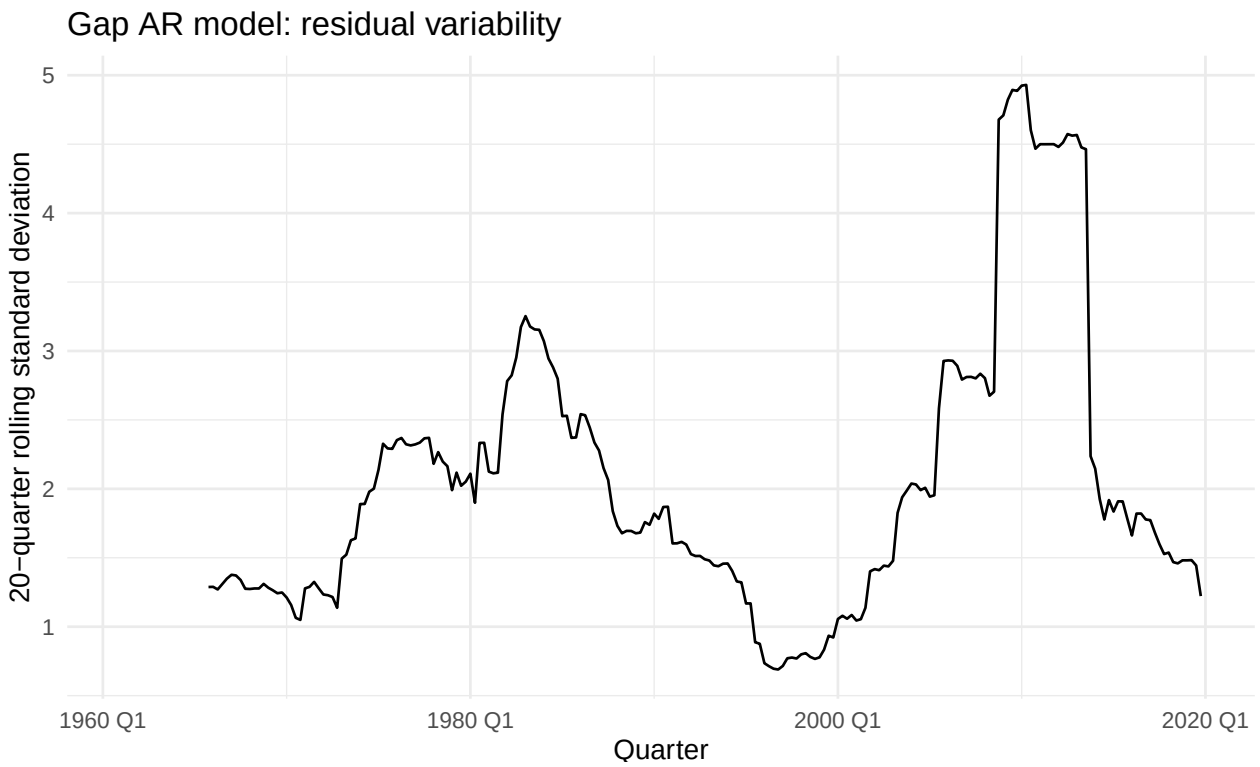
```
    Box-Ljung test

data:  residual_data$residual
X-squared = 11.166, df = 9, p-value = 0.2645
```

Ljung-Box test The Ljung-Box test (at 12 lags) delivers a test statistic of 11.166 with 9 degrees of freedom and a p-value of 0.2644993. Given this result, we fail to reject the null hypothesis of no residual autocorrelation through lag 12. This result provides no strong evidence of remaining serial correlation, suggesting that the fitted gap AR model captures much of the linear serial dependence in the inflation gap.

```r
# residual variability
residual_data <- residual_data |>
  dplyr::mutate(
    rolling_sd = slider::slide_dbl(
      residual,
      sd,
      .before = 19,
      .complete = TRUE
    )
  )

residual_data |>
  ggplot2::ggplot(
    ggplot2::aes(x=Quarter, y = rolling_sd)
) +
  ggplot2::geom_line() +
  tsibble::scale_x_yearquarter() +
  ggplot2::labs(
    title = "Gap AR model: residual variability",
    x = "Quarter",
    y = "20-quarter rolling standard deviation"
  ) +
  ggplot2::theme_minimal()
```



Residual variability The 20-quarter rolling standard deviation indicates that the gap AR model's residual variability changes quite a lot over time. Variability increases during the 1970s and early

1980s and declines throughout the 1990s. It increases again sharply in the 2000s and declines precipitously in the mid-2010s. The largest rolling standard deviation is almost 5, while the smallest is below 1. This suggests that the model's error variance is not stable across the sample. The periods of increased variability may also suggest that a relatively small number of large errors contribute disproportionately to RMSE.

```
# largest errors
residual_data |>
  tibble::as_tibble() |>
  dplyr::mutate(squared_error = residual^2) |>
  dplyr::arrange(dplyr::desc(squared_error)) |>
  dplyr::slice_head(n = 5) |>
  dplyr::select(Quarter, residual, squared_error) |>
  knitr::kable(digits = 3)
```

Quarter	residual	squared_error
2008 Q4	-16.898	285.545
2005 Q3	7.353	54.060
2005 Q4	-6.111	37.346
1986 Q1	-5.337	28.480
1981 Q4	-5.302	28.116

```
squared_errors <- residual_data$residual^2
top5 <- sum(sort(squared_errors, decreasing = TRUE)[1:5]) /
  sum(squared_errors) * 100
```

Largest residuals The largest fitted residuals are generated primarily during the 1980s and 2000s, with the largest in 2008Q4. Because RMSE puts greater weight on large errors (because it squares errors), these observations may contribute disproportionately to the gap AR model's in-sample sum of squared residuals. The five largest errors shown in the table above contribute 37.55% of the overall error. This finding is consistent with the periods of high variation observed in the rolling-standard-deviation plot.

Step 6: Summarize the Stage 2 results